Project Design Phase-II
Technology Stack (Architecture & Stack)

Date	08 Feb 2026
Team ID	LTVIP2026TMIDS91131
Project Name	AutoSage App Using Gemini Flash
Maximum Marks	4 Marks

Technical Architecture:

AutoSage follows a Client–AI Architecture Model consisting of:

1. User Interface Layer (Streamlit Web App)
2. Application Logic Layer (Python Backend)
3. AI Processing Layer (Gemini 2.5 Flash API)
4. Output Rendering Layer
5. Deployment Layer (Local / Streamlit Cloud)

Technical Architecture – AutoSage

AutoSage follows a Client—AI Architecture Model comprised of five layers.

USER

1 User Interface Layer (Streamlit Web App)

AutoSage

+ Upload Vehicle Image

- Upload image in JPG, JPEG, Format
- Validate image format
- Display uploaded image preview

2. Application Logic Layer (Python Backend)

Python

Streamlit session State

- Image processing logic
- Prompt construction

USER

3. AI Processing Layer (Gemini 2.5 Flash API)

Gemini

Google Gemini 2.5 Flash

- Google Gemini 2.5 Flash
- Session state handling & 1l mode Switchiing (Image / asistant)

User> Lnterface

4. Output Rendering Layer

Honda City 2023

- Stramed,,.veetor time ürghtew browser
- Use 'bullet, points – tategoreized
- Maintainchat nistory during session
- Handle API quota errors gracefully

TECHNOLOGY STACK

- Streamlit Web UI
- Python Backend
- Gemini AI API
- Deployment

- Streamlit Web UI
- Python Backend
- Gemini AI API
- Deployment

- Local System (Optonal)
- Streamlit Cloud

Technical Architecture — Streamlit Web UI · Python Backend · Gemini AI API · Deployment

The Deliverable shall include the architectural diagram as below and the information as per the table1 & table 2

Table-1 : Components & Technologies:

S.No	Component	Description	Technology
1	User Interface	Web-based interface for uploading images and interacting with assistant	Streamlit (Python-based Web Framework)
2	Application Logic-1	Image processing logic and prompt construction	Python
3	Application Logic-2	Conversational query processing and structured prompt generation	Python
4	Application Logic-3	Session state handling and UI mode switching (Image / Assistant)	Streamlit Session State
5	File Storage	Temporary image storage during session	In-memory processing (PIL / Local Runtime Memory)
6	External API-1	Multimodal AI processing for image & text analysis	Google Gemini 2.5 Flash API
7	Machine Learning Model	Multimodal Generative AI for vehicle identification and advisory	Gemini 2.5 Flash
8	Infrastructure (Server / Cloud)	Local development and optional cloud deployment	Local System / Streamlit Cloud

Table-2: Application Characteristics:

S.No	Characteristics	Description	Technology
1	Open-Source Frameworks	Application built using open-source libraries and frameworks	Python, Streamlit, Pillow, python-dotenv
2	Security Implementations	Secure API key handling using environment variables; no hardcoded credentials	.env file, Streamlit Secrets, OS Environment Variables
3	Scalable Architecture	Modular architecture separating UI, logic, and AI layers; supports feature expansion	API-Based AI Integration (Gemini)
4	Availability	Web-based deployment accessible via browser; depends on cloud hosting & API availability	Streamlit Cloud / Local Server
5	Performance	Optimized prompt engineering and lightweight UI; average 7–12 sec response time	Gemini Flash (Fast Inference Model)